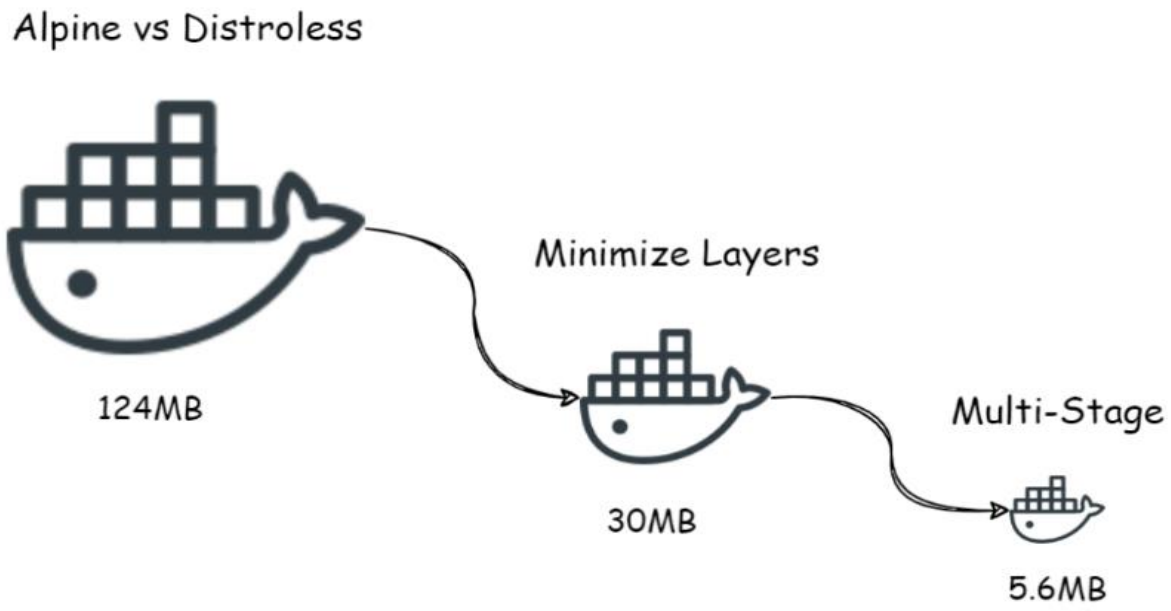
Docker pros are shrinking images by 99%:

The hidden techniques you can’t afford to miss



Shrinking Docker images by 99% sounds impressive, and there are several powerful techniques that professionals use to achieve ultra-lightweight containers. Here are the key strategies:

---

1. Use Minimal Base Images

- Instead of using bloated images like ubuntu or debian, opt for ultra-small base images:
- scratch → The smallest possible image (literally empty).
- alpine → ~5MB and has package management.

diastoles → Google's minimal runtime-only images.

2. Multi-Stage Builds

This technique compiles and builds your application in one stage and then copies only the necessary files into a lightweight runtime image.

Dockerfile

```
# Stage 1: Build
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o app

# Stage 2: Run
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/app .
CMD ["./app"]
```

Result: Drastically reduces image size by stripping out build dependencies.

3. Remove Unnecessary Layers

Each RUN, COPY, or ADD in a Dockerfile creates a layer. Combine commands to minimize layers:

```Dockerfile
RUN apt-get update && apt-get install -y \
    curl \
    && rm -rf /var/lib/apt/lists/*
```

This ensures temporary files are removed within the same layer, reducing image bloat.

4. Use .dockerignore

Exclude unnecessary files (e.g., .git, logs, node_modules) from getting copied into the image.

```plaintext
node_modules
.git
*.log
```

5. Minify Dependencies

- If using Python, use slim versions of images (python:3.11-slim).
- For Node.js, use npm ci instead of npm install to avoid unnecessary dependencies.
- Strip debug symbols from compiled binaries (strip command).

6. Leverage UPX Compression (for binaries)

If your image includes compiled binaries, you can compress them using UPX:

```bash
upx --best mybinary
```

This can reduce binary sizes by 50%+.

7. Use OverlayFS & Deduplication

Docker's layered filesystem (OverlayFS) can reuse common layers across multiple images, reducing storage needs.

8. Enable docker build --squash (Experimental Feature)

Squashing combines multiple layers into one, significantly reducing the final image size.

9. Prefer Static Binaries When Possible

For Go, Rust, and other languages that allow static compilation, build a single executable and use scratch as the base image:

```Dockerfile
FROM scratch
COPY mybinary /
CMD ["/mybinary"]
```

This results in **near-zero overhead**.

10. Remove Unused Packages and Cache

Use package manager flags to prevent installation of unnecessary files:

```
Dockerfile

RUN apk add --no-cache curl
RUN apt-get install -y --no-install-recommends package-name
```

By applying these techniques, you can dramatically reduce your Docker image sizes—sometimes by 99%—leading to faster builds, lower storage costs, and better security